

TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỀ TÀI
XÂY DỰNG ỨNG DỤNG CHAT TRONG MẠNG LAN SỬ
DỤNG GIAO THỨC TCP

Họ và tên: Ngô Thị Cẩm Ly

Mã sinh viên: 22T1020662

Giáo viên hướng dẫn: Nguyễn Quang Hưng

Huế, năm 2026

MỤC LỤC

1. Giới thiệu đề tài	1
2. Mục tiêu đề tài	1
3. Kiến thức sử dụng.....	2
3.1 Socket Programming	2
3.2 Threading.....	2
3.3 Tkinter	3
3.4 Mã hóa Fernet.....	3
3.5 JSON.....	4
3.6 Kiến trúc hệ thống	4
4. Mô hình hệ thống và cấu trúc chương trình	5
4.1. Mô hình hệ thống.....	5
4.2 Cấu trúc thư mục chương trình.....	6
5. Phân tích chức năng hệ thống	7
5.1 Chức năng tìm Server tự động.....	7
5.2 Chức năng đăng nhập	7
5.3 Chức năng gửi tin nhắn.....	8
5.4 Chức năng gửi hình ảnh.....	8
5.5 Chức năng gửi file	8
5.6 Chức năng gửi emoji	8
5.7 Chức năng hiển thị online.....	8
5.8 Chức năng trạng thái đang nhập	9
6. Quy trình hoạt động của hệ thống	9
6.1 Giai đoạn tìm kiếm Server trong mạng LAN	9
6.2 Giai đoạn thiết lập kết nối TCP	10
6.3 Giai đoạn đăng nhập người dùng.....	10
6.4 Giai đoạn truyền tin nhắn	11
6.5 Giai đoạn gửi và nhận hình ảnh.....	12
6.6 Giai đoạn gửi và nhận file	14
6.7 Giai đoạn gửi và nhận emoji.....	15
7. Kết quả thực nghiệm và giao diện demo	16
7.1 Môi trường thực hiện.....	16
7.2 Kết quả giao diện Server	16

7.3 Kết quả giao diện Client	17
7.4 Kết quả tự động tìm Server.....	18
7.5 Kết quả kiểm tra tên đăng nhập trùng.....	18
7.6 Kết quả gửi và nhận tin nhắn.....	19
7.7 Kết quả gửi hình ảnh	19
7.8 Kết quả gửi file	21
7.9 Kết quả gửi emoji	22
7.10 Kết quả hiển thị trạng thái đang nhập.....	23
7.11 Kết quả chat riêng tư.....	23
7.12 Đánh giá kết quả	24
8. Ưu điểm	24
9. Nhược điểm	25
10. Hướng phát triển	25
11. Kết luận.....	26

DANH MỤC HÌNH ẢNH

Hình 3.1: Quy trình hoạt động của mã hóa Fernet	3
Hình 4.1: Sơ đồ hoạt động của hệ thống chat.....	6
Hình 7.1. Giao diện Server khi hệ thống hoạt động	16
Hình 7.2: Giao diện Server khi có các Client hoạt động	17
Hình 7.3. Giao diện chính của Client	17
Hình 7.4. Kết quả dò tìm Server tự động.....	18
Hình 7.5. Kết quả kiểm tra tên đăng nhập bị trùng	18
Hình 7.6. Kết quả gửi tin nhắn văn bản.....	19
Hình 7.7: Kết quả hiển thị hình ảnh	19
Hình 7.8: Kết quả phóng to hình ảnh	20
Hình 7.9: Kết quả tải và lưu hình ảnh.....	20
Hình 7.10: Kết quả file được lưu vào mục received.....	21
Hình 7.11: Kết quả hiển thị file	21
Hình 7.12: Kết quả tải file xuống	22
Hình 7.13. Kết quả gửi emoji	22
Hình 7.14. Kết quả hiển thị trạng thái đang nhập.....	23

1. Giới thiệu đề tài

Trong bối cảnh công nghệ thông tin phát triển, nhu cầu giao tiếp và trao đổi dữ liệu qua mạng ngày càng phổ biến. Đề tài xây dựng ứng dụng chat trong mạng LAN sử dụng giao thức TCP nhằm tạo hệ thống trò chuyện giữa nhiều người dùng trong cùng mạng nội bộ bằng ngôn ngữ Python.

Ứng dụng được phát triển theo mô hình Client–Server, sử dụng thư viện socket, threading và tkinter. Hệ thống hỗ trợ các chức năng như gửi tin nhắn, hình ảnh, tệp, emoji, hiển thị người dùng online, trạng thái đang nhập, chat riêng tư, lưu lịch sử trò chuyện, avatar, tự động tìm Server bằng UDP Broadcast và mã hóa dữ liệu.

Thông qua đề tài, giúp vận dụng kiến thức về lập trình mạng, giao thức TCP/IP, xử lý đa luồng và xây dựng giao diện ứng dụng thực tế.

2. Mục tiêu đề tài

- Xây dựng ứng dụng chat theo mô hình Client–Server bằng ngôn ngữ Python nhằm hỗ trợ giao tiếp giữa nhiều người dùng trong cùng mạng LAN.
- Sử dụng giao thức TCP Socket để đảm bảo dữ liệu được truyền ổn định, chính xác và hạn chế mất gói tin trong quá trình trao đổi thông tin.
- Thiết kế giao diện đồ họa trực quan bằng thư viện Tkinter, giúp người dùng dễ dàng thao tác và sử dụng ứng dụng.
- Hỗ trợ nhiều Client kết nối đồng thời đến Server, cho phép nhiều người dùng chat cùng lúc trong hệ thống.
- Xây dựng chức năng gửi và nhận nhiều loại dữ liệu khác nhau gồm: Tin nhắn văn bản, hình ảnh, file tài liệu, emoji.
- Hiển thị danh sách người dùng đang online theo thời gian thực để hỗ trợ quản lý và nhận biết trạng thái kết nối của các Client.
- Tích hợp chức năng tự động dò tìm địa chỉ IP của Server trong mạng LAN bằng UDP Broadcast, giúp người dùng không cần nhập thủ công IP Server.

- Áp dụng cơ chế mã hóa dữ liệu bằng Fernet Encryption nhằm tăng tính bảo mật cho nội dung trao đổi giữa Client và Server.
- Hiện thị trạng thái “đang nhập...” (typing) để nâng cao tính tương tác khi chat.
- Hỗ trợ lưu lịch sử tin nhắn, hiện thị thời gian gửi tin nhắn và quản lý kết nối/ngắt kết nối của người dùng trong hệ thống.

3. Kiến thức sử dụng

3.1 Socket Programming

Socket Programming là kỹ thuật cho phép các tiến trình trên các thiết bị khác nhau giao tiếp với nhau thông qua mạng máy tính.

Trong đề tài này, thư viện socket của Python được dùng để thiết lập kết nối giữa hai thành phần chính:

- TCP Socket (SOCK_STREAM): Đảm nhận luồng dữ liệu chat chính, truyền nhận file và hình ảnh. Giao thức này yêu cầu thiết lập kết nối (3-way handshake), đảm bảo dữ liệu đến đích nguyên vẹn và đúng trình tự.
- UDP Socket (SOCK_DGRAM): Phục vụ cho chức năng Discovery. Client gửi gói tin Broadcast lên mạng nội bộ, Server lắng nghe và phản hồi IP mà không cần duy trì kết nối liên tục, giúp tối ưu hóa băng thông.

3.2 Threading

Kỹ thuật đa luồng (import threading) giải quyết bài toán nghẽn luồng (blocking) trong lập trình mạng và GUI. Khi một Socket hoặc giao diện đồ họa chờ dữ liệu, luồng chính sẽ bị khóa nếu không có đa luồng.

Ứng dụng phân tách chương trình thành các luồng độc lập:

- Luồng chấp nhận kết nối mới tại Server (accept_connections).
- Luồng lắng nghe tin nhắn từ từng Client cụ thể (handle_client).
- Luồng phản hồi tín hiệu tìm kiếm UDP (udp_broadcast_responder).
- Luồng cập nhật giao diện GUI phía Client luôn giữ trạng thái mượt mà khi truyền nhận file kích thước lớn.

3.3 Tkinter

Tkinter là thư viện tạo giao diện đồ họa (GUI) có sẵn trong Python. Thư viện này được sử dụng để xây dựng giao diện cho ứng dụng chat.

Các chức năng GUI được xây dựng bằng Tkinter gồm:

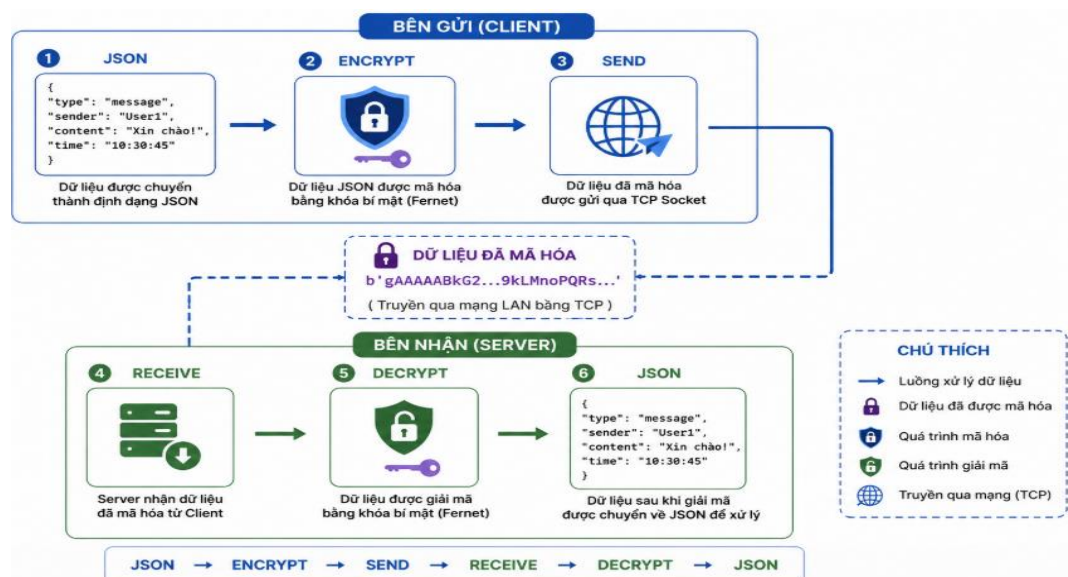
- Tạo cửa sổ chat chính.
- Hiển thị thời gian gửi tin nhắn.
- Hiển thị danh sách người dùng online.
- Tạo ô nhập nội dung chat.
- Tạo các nút: Gửi tin nhắn, gửi ảnh, gửi file, gửi emoji.
- Hiển thị trạng thái “đang nhập...”.
- Hiển thị avatar người dùng.

Ngoài ra, đề tài còn sử dụng: `from tkinter import scrolledtext`.

3.4 Mã hóa Fernet

Để tăng tính bảo mật cho dữ liệu truyền trên mạng, ứng dụng sử dụng thư viện Fernet thuộc package cryptography. Thư viện sử dụng: `from cryptography.fernet import Fernet`.

Quy trình hoạt động:



Hình 3.1: Quy trình hoạt động của mã hóa Fernet

Cụ thể:

1. Dữ liệu chat được chuyển thành JSON.
2. JSON được mã hóa bằng khóa bí mật dùng chung.
3. Dữ liệu mã hóa được gửi qua TCP Socket.
4. Client/Server nhận dữ liệu.
5. Dữ liệu được giải mã.
6. Chuyển lại thành JSON để xử lý.

Việc sử dụng Fernet giúp nâng cao tính an toàn và bảo mật cho ứng dụng chat Client–Server.

3.5 JSON

JSON được sử dụng để đóng gói dữ liệu truyền giữa Client và Server.

Các loại dữ liệu: Tin nhắn văn bản, hình ảnh, file, emoji, danh sách online, trạng thái typing.

Việc sử dụng JSON giúp dữ liệu dễ quản lý, xử lý và mở rộng thêm chức năng sau này.

3.6 Kiến trúc hệ thống

Hệ thống được xây dựng theo kiến trúc Client–Server gồm một Server trung tâm và nhiều Client kết nối thông qua mạng LAN nội bộ. Trong mô hình này, Server đóng vai trò quản lý và điều phối toàn bộ hoạt động trao đổi dữ liệu giữa các Client.

Server chịu trách nhiệm:

- Tiếp nhận và quản lý kết nối từ nhiều Client đồng thời
- Xử lý dữ liệu nhận được từ các Client
- Broadcast tin nhắn đến các Client khác trong hệ thống
- Quản lý danh sách người dùng online
- Kiểm tra tên đăng nhập và trạng thái kết nối
- Xử lý các chức năng như chat riêng tư, typing status và truyền file

Client đóng vai trò là chương trình phía người dùng với các chức năng:

- Hiển thị giao diện chat
- Gửi và nhận dữ liệu từ Server
- Hiển thị tin nhắn, hình ảnh, file và emoji
- Hiển thị danh sách người dùng online
- Tương tác với người dùng thông qua giao diện GUI

Trong hệ thống, giao thức TCP được sử dụng cho quá trình truyền dữ liệu chính nhằm đảm bảo độ tin cậy và tính toàn vẹn của dữ liệu. Bên cạnh đó, UDP Broadcast được sử dụng để hỗ trợ chức năng tự động dò tìm Server trong mạng LAN.

Dữ liệu truyền giữa Client và Server được đóng gói dưới dạng JSON và mã hóa bằng Fernet Encryption trước khi gửi qua Socket nhằm tăng tính bảo mật cho hệ thống.

4. Mô hình hệ thống và cấu trúc chương trình

4.1. Mô hình hệ thống

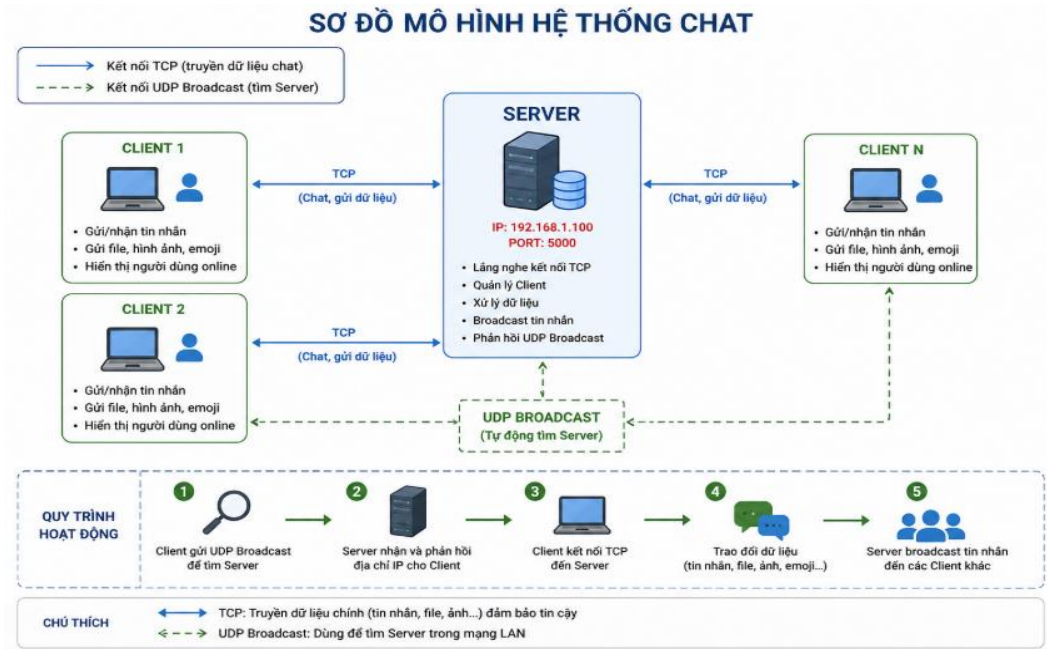
Hệ thống được xây dựng theo mô hình Client–Server.

Server đóng vai trò trung tâm tiếp nhận kết nối, xử lý dữ liệu và truyền tin nhắn đến các Client đang online trong mạng LAN.

Các Client sẽ kết nối đến Server thông qua giao thức TCP để thực hiện: Gửi và nhận tin nhắn, gửi hình ảnh, gửi file, gửi emoji, nhận danh sách người dùng online.

Ngoài ra, hệ thống còn sử dụng UDP Broadcast để hỗ trợ chức năng tự động tìm địa chỉ IP của Server trong mạng LAN, giúp người dùng không cần nhập thủ công IP Server khi kết nối.

Sơ đồ mô hình hệ thống:



Hình 4.1: Sơ đồ hoạt động của hệ thống chat

Nguyên lý hoạt động

1. Khi khởi động, Client sẽ gửi gói tin UDP Broadcast trong mạng LAN để tìm Server.
2. Server nhận tín hiệu dò tìm và phản hồi lại địa chỉ IP của mình.
3. Sau khi tìm được IP, Client thực hiện kết nối TCP tới Server.
4. Server tiếp nhận nhiều Client đồng thời bằng đa luồng (Threading).
5. Mọi dữ liệu chat sẽ được: Chuyển sang JSON ,mã hóa bằng Fernet ,gửi qua TCP Socket
6. Server nhận dữ liệu, giải mã và phát lại cho các Client khác trong hệ thống.

4.2 Cấu trúc thư mục chương trình

Chương trình được tổ chức thành 3 thư mục chính:

Thư mục Client: Chứa chương trình phía người dùng, gồm giao diện và các chức năng gửi nhận dữ liệu.

- client_gui.py: giao diện chính phía Client.
- protocol.py: xử lý đóng gói, giải mã dữ liệu.

- avatars/: ảnh đại diện.
- emoji/: biểu tượng cảm xúc.
- received/: lưu file nhận được.
- chat_history.txt: lưu lịch sử trò chuyện.

Thư mục Server: Chứa chương trình trung tâm quản lý kết nối và xử lý dữ liệu.

- server_gui.py: giao diện quản lý Server.
- client_handler.py: xử lý từng Client.
- protocol.py: giao thức truyền dữ liệu.
- utils.py: các hàm hỗ trợ.

Thư mục common: Chứa các thành phần dùng chung giữa Client và Server.

- protocol.py: đóng gói dữ liệu, mã hóa, giải mã.

5. Phân tích chức năng hệ thống

5.1 Chức năng tìm Server tự động

Client tự động tìm địa chỉ IP của Server trong mạng LAN bằng UDP Broadcast:

- Không cần nhập IP thủ công.
- Kết nối nhanh.
- Dễ sử dụng.

5.2 Chức năng đăng nhập

Khi chạy chương trình, người dùng nhập tên đăng nhập. Hệ thống tiến hành kiểm tra:

- Không được để trống.
- Không được chứa khoảng trắng.
- Không được trùng tên.

Sau khi đăng nhập thành công, hệ thống sẽ:

- Hiện thị danh sách các người dùng online.
- Hiện thị tên trên cửa sổ chat.

5.3 Chức năng gửi tin nhắn

Người dùng nhập nội dung và gửi bằng nút Gửi hoặc phím Enter:

- Gửi tin nhắn thời gian thực.
- Hiển thị thời gian gửi tin nhắn.
- Hiển thị theo tên người gửi.
- Tin nhắn được căn trái/phải theo người gửi.

5.4 Chức năng gửi hình ảnh

Pillow được sử dụng để xử lý ảnh, người dùng chọn ảnh từ máy tính:

- Nén ảnh trước khi gửi.
- Gửi ảnh qua TCP.
- Hiển thị ảnh trong khung chat.
- Cho phép mở ảnh lớn để xem.
- Tải ảnh về máy.

5.5 Chức năng gửi file

Người dùng có thể gửi: txt, pdf, docx, zip, rar, pptx. Sau khi nhận file:

- File lưu trong thư mục received.
- Hiển thị tên file.
- Có thể tải về.

Giới hạn dung lượng tối đa 5MB.

5.6 Chức năng gửi emoji

Người dùng chọn emoji từ thư mục emoji:

- Gửi emoji dạng ảnh.
- Hiển thị trong chat.
- Đồng bộ giữa các Client.

5.7 Chức năng hiển thị online

Server quản lý danh sách người dùng đang hoạt động. Client sẽ hiển thị:

- Tên người dùng.

- Trạng thái online.
- Số lượng online.

5.8 Chức năng trạng thái đang nhập

Khi người dùng gõ tin nhắn:

- Client gửi tín hiệu typing.
- Server chuyển tiếp tín hiệu đến các Client khác.
- Hiện thị “đang nhập...” trên giao diện chat.

6. Quy trình hoạt động của hệ thống

6.1 Giai đoạn tìm kiếm Server trong mạng LAN

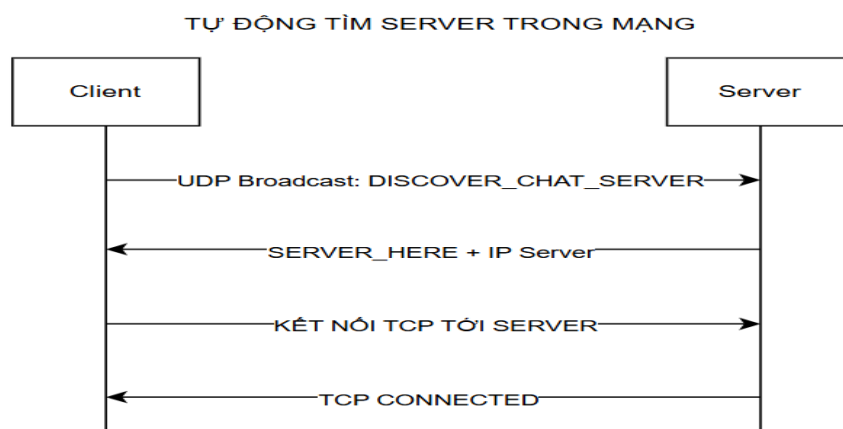
Khi khởi động ứng dụng, Client sẽ tự động gửi gói tin UDP Broadcast trong mạng LAN để tìm Server.

Client gửi thông điệp: *DISCOVER_CHAT_SERVER*

Server sẽ lắng và phản hồi: *SERVER_HERE*

Sau khi nhận được phản hồi, Client tiến hành lưu địa chỉ IP của Server để chuẩn bị thiết lập kết nối TCP.

Quy trình hoạt động



Cơ chế UDP Broadcast giúp:

- Tự động phát hiện Server trong mạng LAN.
- Không cần nhập thủ công địa chỉ IP.

- Tăng tính thuận tiện cho người dùng.

6.2 Giai đoạn thiết lập kết nối TCP

Sau khi xác định được địa chỉ IP của Server, Client sẽ thực hiện kết nối TCP tới Server thông qua Socket Programming.

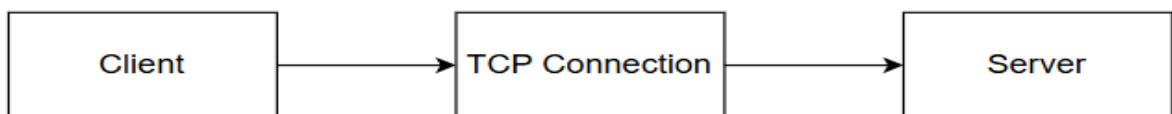
```
client.connect((HOST, PORT))
```

Giao thức TCP được lựa chọn vì:

- Đảm bảo dữ liệu truyền chính xác.
- Dữ liệu đến đúng thứ tự.
- Hạn chế mất gói tin trong quá trình truyền tải.

Server sử dụng cơ chế đa luồng để cho phép nhiều Client kết nối đồng thời mà không làm gián đoạn hệ thống.

Mô hình kết nối:



6.3 Giai đoạn đăng nhập người dùng

Người dùng nhập tên đăng nhập và lựa chọn avatar trước khi tham gia hệ thống chat.

Client sẽ gửi thông tin đăng nhập tới Server dưới dạng JSON:

```
send_data(client, {  
    "username": username,  
    "avatar": avatar_hex  
})
```

Server tiến hành kiểm tra:

- Tên đăng nhập không được để trống.
- Không chứa khoảng trắng.
- Không trùng với người dùng đang online.

Nếu tên không hợp lệ, Server sẽ gửi phản hồi lỗi như:

```
# Chờ phản hồi từ server
data = receive_data(client)

if data.get("type") == "username_taken":
    messagebox.showerror(
        "Tên đã tồn tại",
        "Tên này đã được người khác sử dụng. Vui lòng nhập tên khác."
    )
    continue
elif data.get("type") == "invalid_username":
    messagebox.showerror(
        "Tên không hợp lệ",
        "Tên không được chứa khoảng trắng hoặc rỗng."
    )
    continue
else:
    window.title(f"Client Chat - {username}")
    break
```

Khi đăng nhập thành công:

- Người dùng được thêm vào danh sách online.
- Avatar hiển thị trong giao diện chat.
- Người dùng có thể bắt đầu gửi dữ liệu.

Quy trình hoạt động



6.4 Giai đoạn truyền tin nhắn

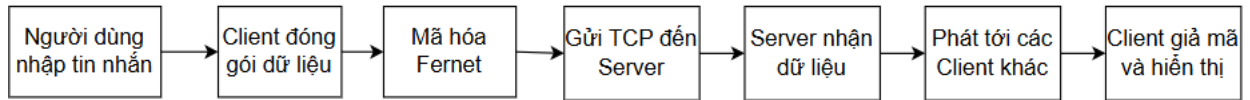
Khi người dùng gửi tin nhắn sẽ được đóng gói dưới dạng JSON trước khi truyền đi.

```
data = {
    "type": "text",
    "msg": formatted_msg_send,
    "from": username,
    "avatar": avatar_hex
}
```

Quá trình xử lý dữ liệu gồm các bước:

1. Chuyển dữ liệu sang định dạng JSON.
2. Mã hóa dữ liệu bằng Fernet.
3. Gửi dữ liệu qua TCP Socket.
4. Server tiếp nhận và giải mã dữ liệu.
5. Server broadcast tới các Client khác.
6. Client nhận dữ liệu và hiển thị lên giao diện GUI.

Quy trình truyền tin nhắn:



Khi nhận được tin nhắn:

- Nếu là tin nhắn của chính mình → hiển thị bên phải.
- Nếu là tin nhắn từ người khác → hiển thị bên trái.

Hệ thống sử dụng giao diện dạng “chat bubble” tương tự các ứng dụng nhắn tin hiện đại nhằm tăng tính trực quan và dễ sử dụng. Ngoài nội dung tin nhắn, giao diện còn hiển thị: Thời gian gửi, avatar người gửi, tên người gửi.

Việc căn lề trái/phải giúp người dùng dễ phân biệt tin nhắn gửi và nhận.

6.5 Giai đoạn gửi và nhận hình ảnh

Khi người dùng chọn hình ảnh để gửi, Client sẽ đọc dữ liệu ảnh và thực hiện xử lý trước khi truyền tới Server. Quá trình xử lý gồm:

1. Đọc dữ liệu hình ảnh.
2. Tối ưu kích thước ảnh.
3. Chuyển dữ liệu ảnh sang dạng HEX.
4. Đóng gói dữ liệu dưới dạng JSON.
5. Mã hóa dữ liệu bằng Fernet.
6. Gửi dữ liệu qua TCP Socket.

Ví dụ cấu trúc dữ liệu:

```
data = {  
  "type": "image",  
  "from": username,  
  "filename": filename,  
  "content": content  
}
```

Sau khi nhận dữ liệu:

- Server giải mã và broadcast tới các Client khác.
- Client nhận dữ liệu và hiển thị hình ảnh trong cửa sổ chat.

Code xử lý nén ảnh:

```
image.save(  
    img_bytes,  
    format="JPEG",  
    quality=70,  
    optimize=True  
)
```

Việc nén ảnh giúp:

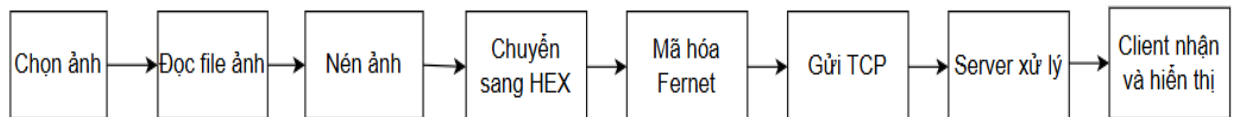
- Giảm dung lượng truyền tải.
- Tăng tốc độ gửi ảnh.
- Tránh gây quá tải mạng LAN.

Sau khi xử lý, dữ liệu ảnh sẽ được chuyển sang HEX trước khi gửi:

```
content = img_bytes.getvalue().hex()
```

Server nhận ảnh và phát lại cho các Client khác đang kết nối.

Quy trình gửi ảnh:



Sau khi nhận ảnh:

- Ảnh sẽ hiển thị trực tiếp trong cửa sổ chat.
- Người dùng có thể nhấn vào ảnh để xem kích thước lớn.
- Hỗ trợ tải ảnh về máy tính.

Hệ thống sử dụng thư viện Pillow (PIL) để xử lý ảnh:

```
from PIL import Image, ImageTk, ImageOps
```

Ngoài ra, ảnh trong khung chat cũng được căn trái/phải tương tự tin nhắn văn bản:
Ảnh của mình hiển thị bên phải, ảnh người khác hiển thị bên trái.

Điều này giúp giao diện đồng bộ và dễ quan sát hơn trong quá trình trò chuyện.

6.6 Giai đoạn gửi và nhận file

Khi người dùng lựa chọn file để gửi, hệ thống sẽ đọc dữ liệu file dưới dạng nhị phân (binary) trước khi truyền qua mạng. Quá trình xử lý gồm:

1. Đọc dữ liệu file.
2. Chuyển dữ liệu sang dạng HEX.
3. Đóng gói dữ liệu dưới dạng JSON.
4. Mã hóa bằng Fernet.
5. Gửi tới Server qua TCP Socket.

Ví dụ đoạn code đọc file:

```
with open(path, "rb") as f:  
    content = f.read().hex()
```

Dữ liệu file sau khi đọc sẽ được đóng gói dưới dạng JSON:

```
data = {  
    "type": "file",  
    "from": username,  
    "filename": filename,  
    "content": content  
}
```

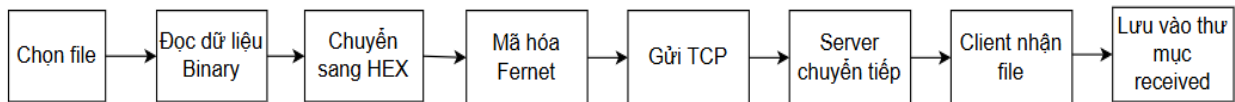
Server sẽ tiếp nhận dữ liệu và phát lại tới các Client khác đang kết nối. Khi Client nhận file:

- File sẽ được lưu vào thư mục received.
- Tên file được hiển thị trực tiếp trong cửa sổ chat.
- Người dùng có thể nhấn vào tên file để tải về máy.

Để tránh quá tải mạng và giảm nguy cơ treo ứng dụng, hệ thống giới hạn kích thước file tối đa là 5MB. Code kiểm tra kích thước file:

```
MAX_FILE_SIZE = 5 * 1024 * 1024  
  
if os.path.getsize(path) > MAX_FILE_SIZE:  
    messagebox.showwarning(  
        "File quá lớn",  
        "Chỉ cho phép file dưới 5MB."  
    )  
    return
```

Quy trình gửi file:



6.7 Giai đoạn gửi và nhận emoji

Khi người dùng lựa chọn emoji trên giao diện chat, Client sẽ đóng gói dữ liệu emoji trước khi gửi tới Server thông qua giao thức TCP.

Emoji được xử lý tương tự tin nhắn văn bản nhằm đảm bảo khả năng hiển thị đồng bộ giữa các Client trong hệ thống.

```
message = {  
  "type": "emoji",  
  "from": username,  
  "content": hex_data  
}
```

Quá trình xử lý dữ liệu emoji gồm:

1. Người dùng chọn emoji từ giao diện
2. Client đóng gói dữ liệu dưới dạng JSON
3. Dữ liệu được mã hóa bằng Fernet
4. Gửi tới Server thông qua TCP Socket
5. Server tiếp nhận và broadcast tới các Client khác
6. Client nhận dữ liệu và hiển thị emoji trong cửa sổ chat

Sau khi nhận dữ liệu:

- Emoji hiển thị trực tiếp trong nội dung chat.
- Các Client được đồng bộ dữ liệu theo thời gian thực.
- Emoji hiển thị cùng chat bubble nhằm đảm bảo giao diện thống nhất.

Quy trình gửi emoji:



7. Kết quả thực nghiệm và giao diện demo

7.1 Môi trường thực hiện

Hệ thống được triển khai và kiểm thử trong mạng LAN nội bộ bằng ngôn ngữ Python. Chương trình được chạy trên nhiều máy tính cùng kết nối một mạng để mô phỏng quá trình trao đổi dữ liệu giữa Server và các Client.

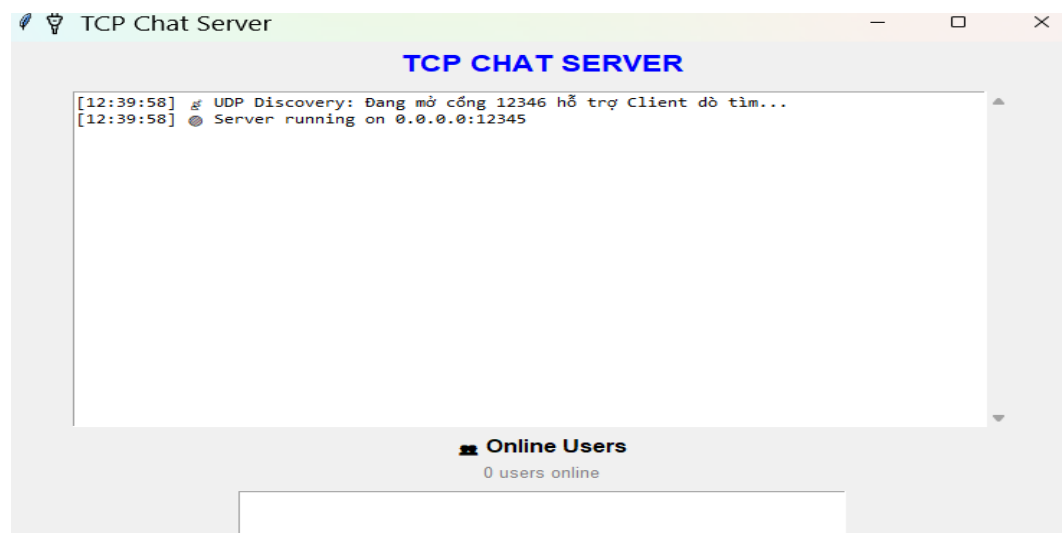
Thông số môi trường:

- Ngôn ngữ lập trình: Python 3.x.
- Thư viện sử dụng: socket, threading, tkinter, PIL.
- Giao thức sử dụng: TCP và UDP Broadcast.
- Mô hình: Client – Server.
- Hệ điều hành thử nghiệm: Windows.
- Mạng thử nghiệm: LAN nội bộ cùng WiFi.

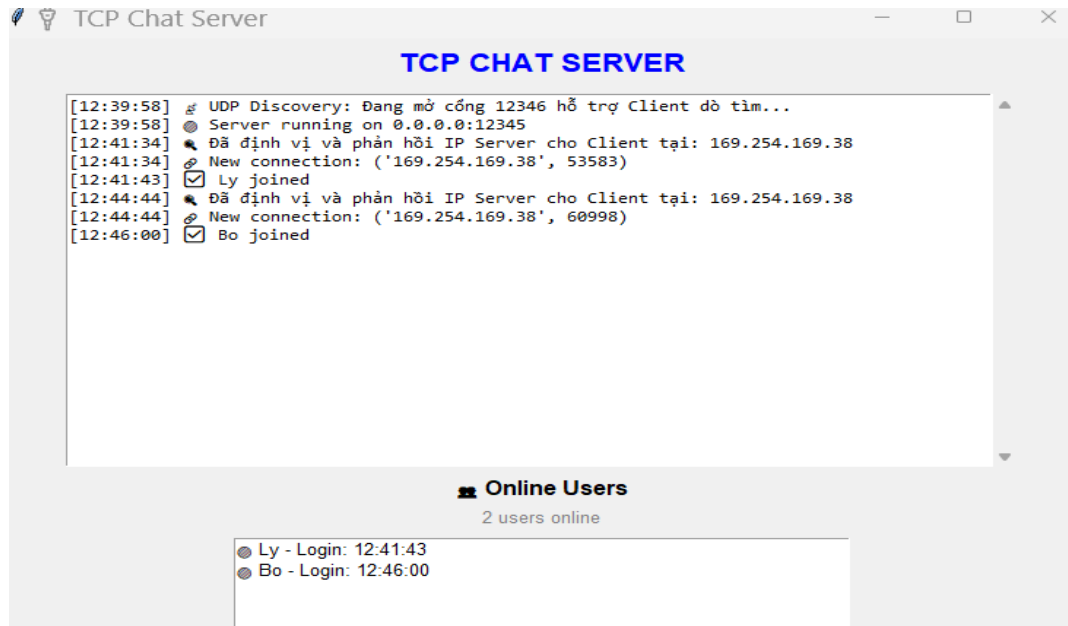
7.2 Kết quả giao diện Server

Giao diện Server hiển thị quá trình hoạt động của hệ thống, bao gồm:

- Danh sách người dùng đang online.
- Thời gian đăng nhập.
- Nhật ký hoạt động.
- Trạng thái kết nối Client.
- Quá trình gửi/nhận dữ liệu.



Hình 7.1. Giao diện Server khi hệ thống hoạt động

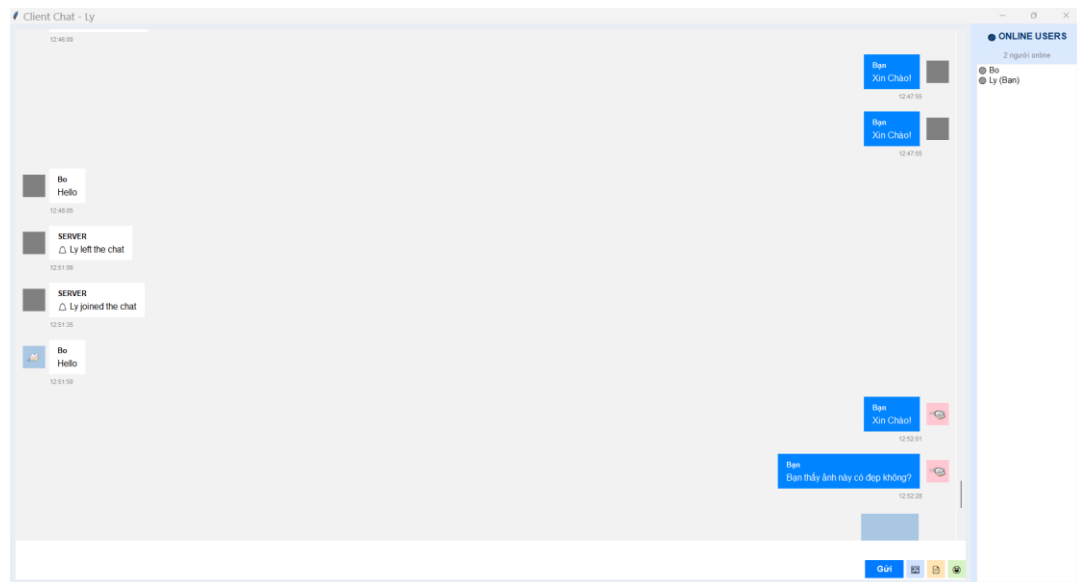


Hình 7.2: Giao diện Server khi có các Client hoạt động

7.3 Kết quả giao diện Client

Giao diện Client được xây dựng bằng Tkinter với các thành phần:

- Khung hiển thị tin nhắn, ô nhập nội dung.
- Nút gửi tin nhắn, gửi ảnh, gửi file.
- Nút chọn và gửi emoji.
- Danh sách online.
- Trạng thái đang nhập.



Hình 7.3. Giao diện chính của Client

7.4 Kết quả tự động tìm Server

Khi Client khởi động, chương trình tự động gửi tín hiệu UDP Broadcast để dò tìm Server trong mạng LAN. Kết quả:

- Client tự nhận địa chỉ IP Server. Không cần nhập IP thủ công. Kết nối thuận tiện hơn.

```
E:\THUCTAP\Client>py client_gui.py
Đang quét tìm Server mạng LAN...
Đã tìm thấy Server tại IP: 169.254.169.38
█
```

Hình 7.4. Kết quả dò tìm Server tự động

7.5 Kết quả kiểm tra tên đăng nhập trùng

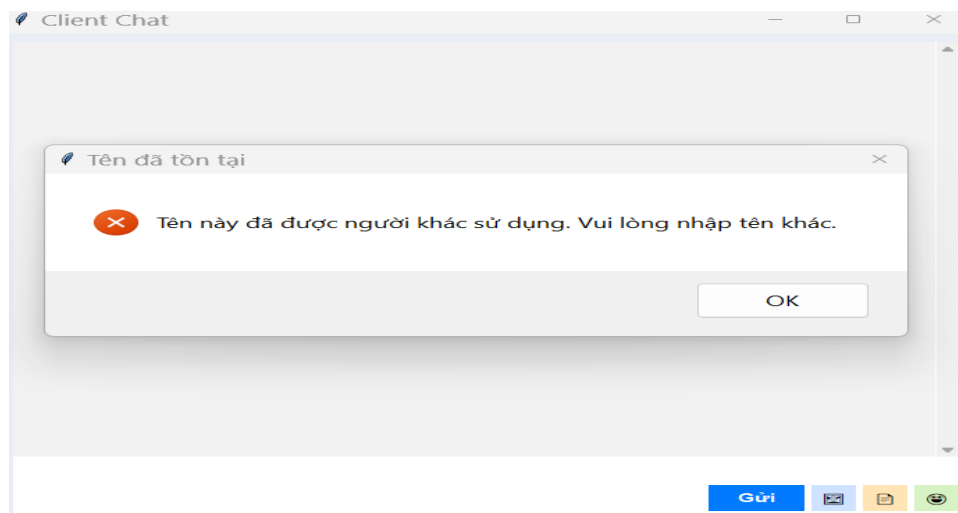
Hệ thống có chức năng kiểm tra tên đăng nhập nhằm tránh nhiều người dùng sử dụng cùng một tên trong phiên chat.

Khi Client gửi tên đăng nhập đến Server, hệ thống sẽ thực hiện kiểm tra:

- Tên không được để trống, không chứa khoảng trắng, không được trùng với người dùng đang online.

Nếu phát hiện tên đã tồn tại, Server sẽ gửi thông báo lỗi về Client. Người dùng phải nhập tên khác mới có thể đăng nhập thành công. Chức năng này giúp:

- Tránh nhầm lẫn giữa các người dùng. Quản lý danh sách online chính xác. Đảm bảo định danh riêng cho mỗi Client.

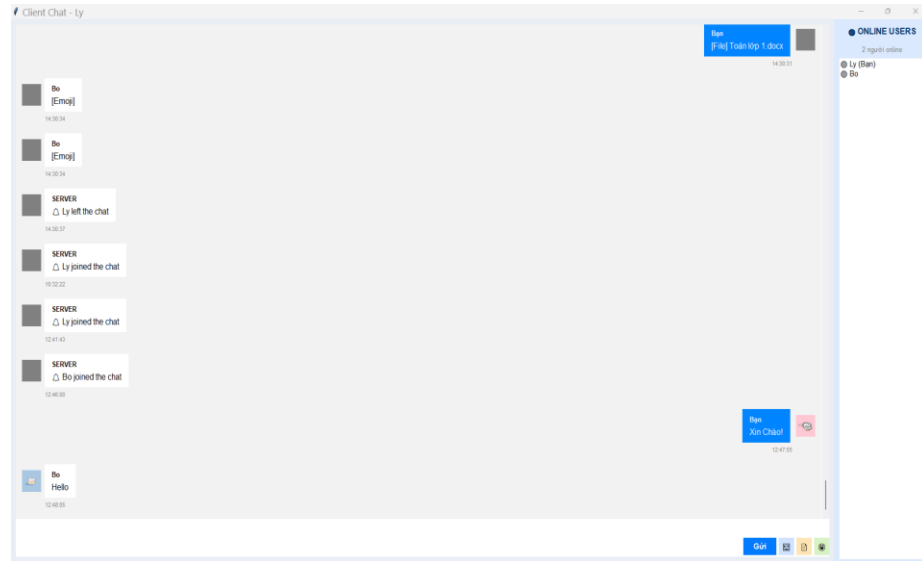


Hình 7.5. Kết quả kiểm tra tên đăng nhập bị trùng

7.6 Kết quả gửi và nhận tin nhắn

Ứng dụng cho phép nhiều người dùng gửi tin nhắn đồng thời trong mạng LAN. Tin nhắn được hiển thị theo thời gian thực với:

- Tên người gửi, thời gian gửi, avatar, căn trái/phải theo người gửi.

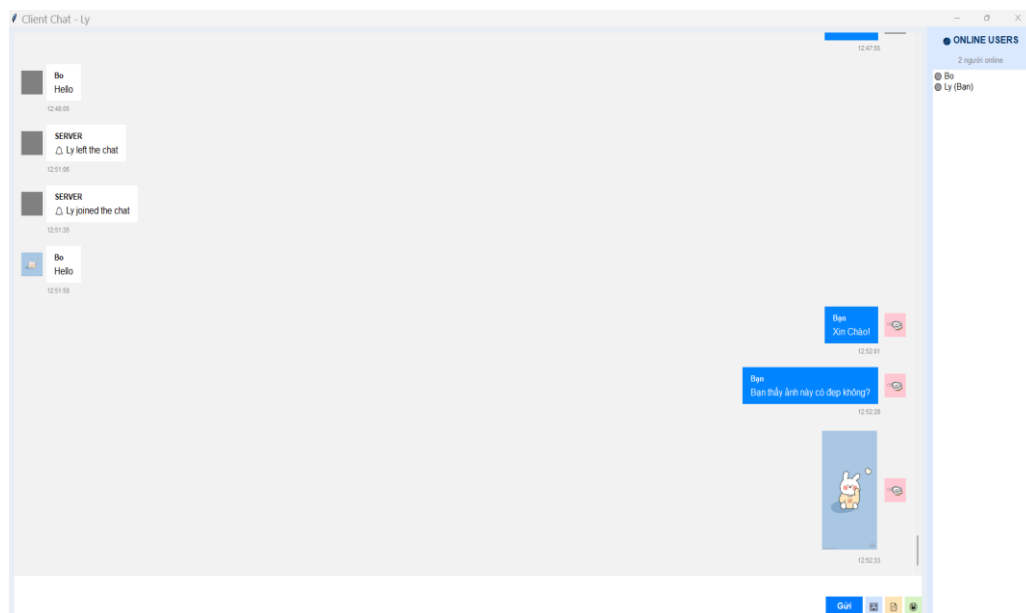


Hình 7.6. Kết quả gửi tin nhắn văn bản

7.7 Kết quả gửi hình ảnh

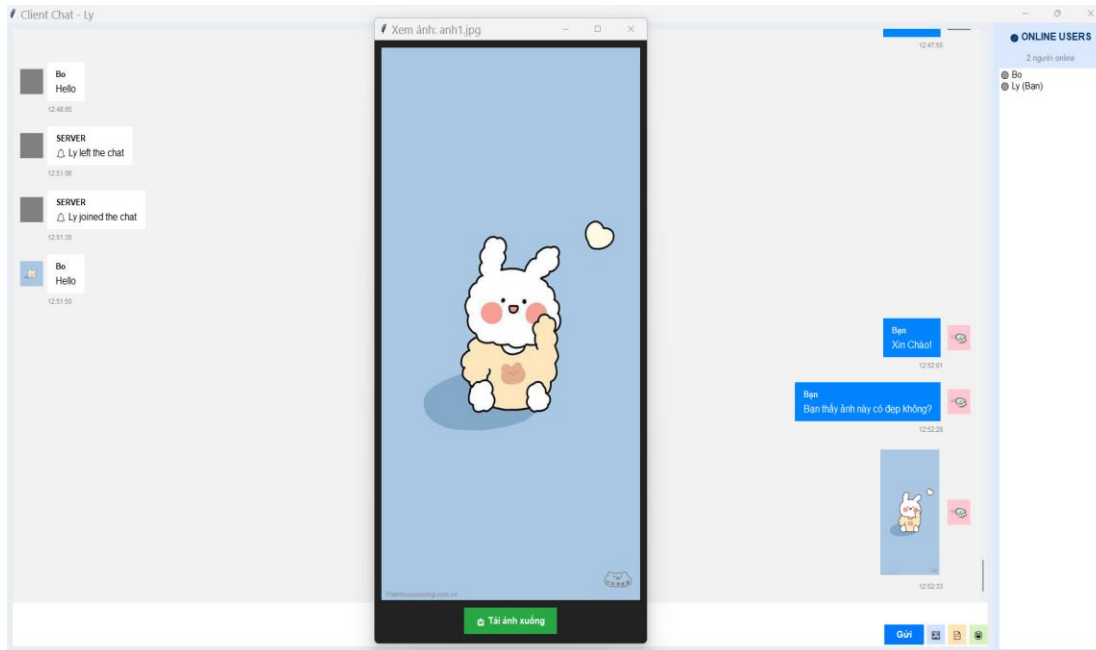
Người dùng có thể chọn ảnh từ máy tính và gửi trực tiếp qua ứng dụng. Sau khi gửi:

- Ảnh hiển thị ngay trên cửa sổ chat:



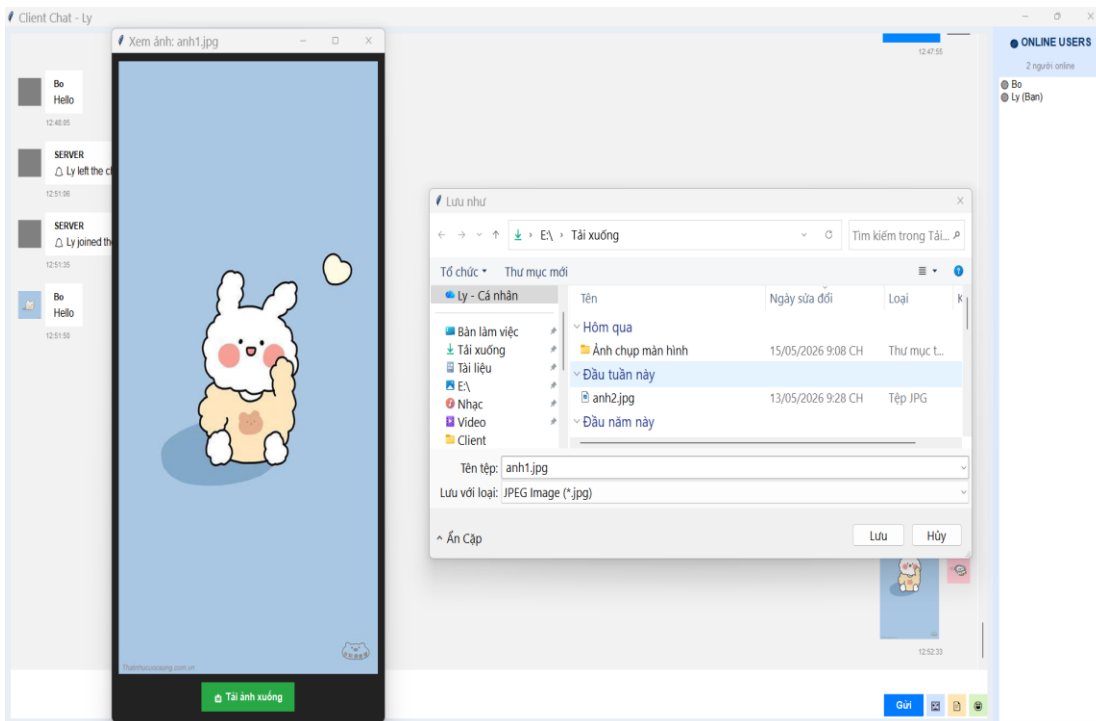
Hình 7.7: Kết quả hiển thị hình ảnh

- Có thể bấm xem ảnh lớn:



Hình 7.8: Kết quả phóng to hình ảnh

- Có thể tải ảnh xuống:

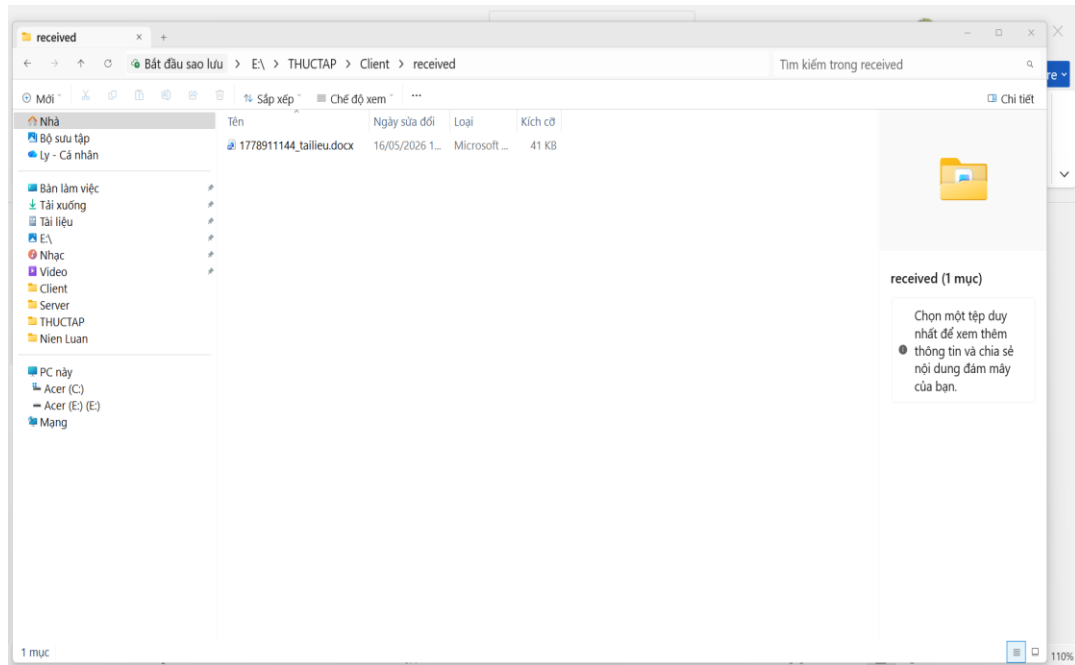


Hình 7.9: Kết quả tải và lưu hình ảnh

7.8 Kết quả gửi file

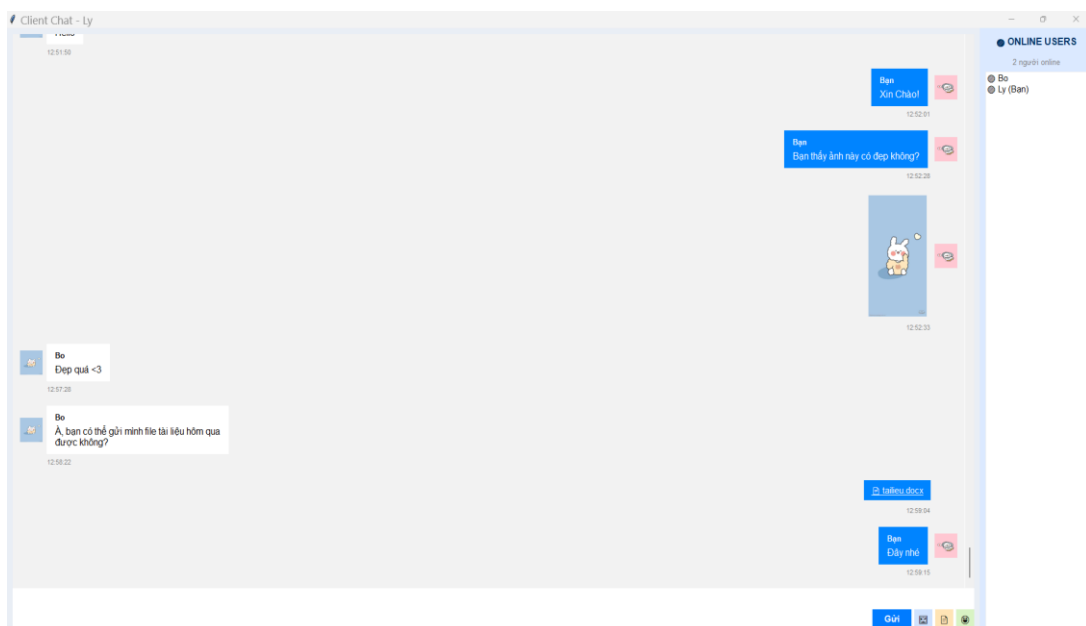
Ứng dụng hỗ trợ truyền file tài liệu nhiều định dạng. Sau khi nhận:

- File lưu vào thư mục received:



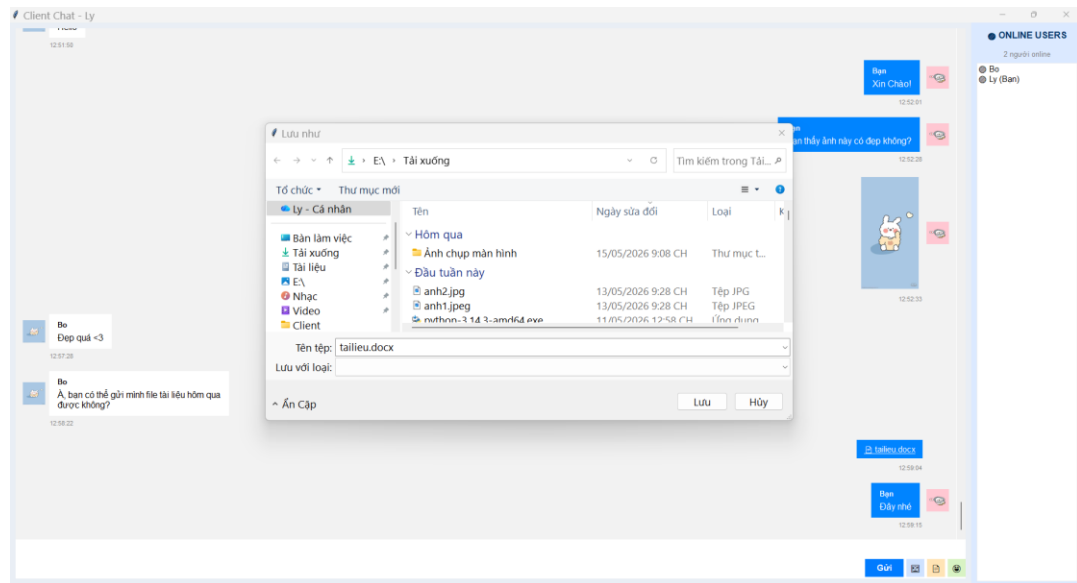
Hình 7.10: Kết quả file được lưu vào mục received

- Hiện thị tên file trên giao diện:



Hình 7.11: Kết quả hiển thị file

- Có thể bấm tải xuống:

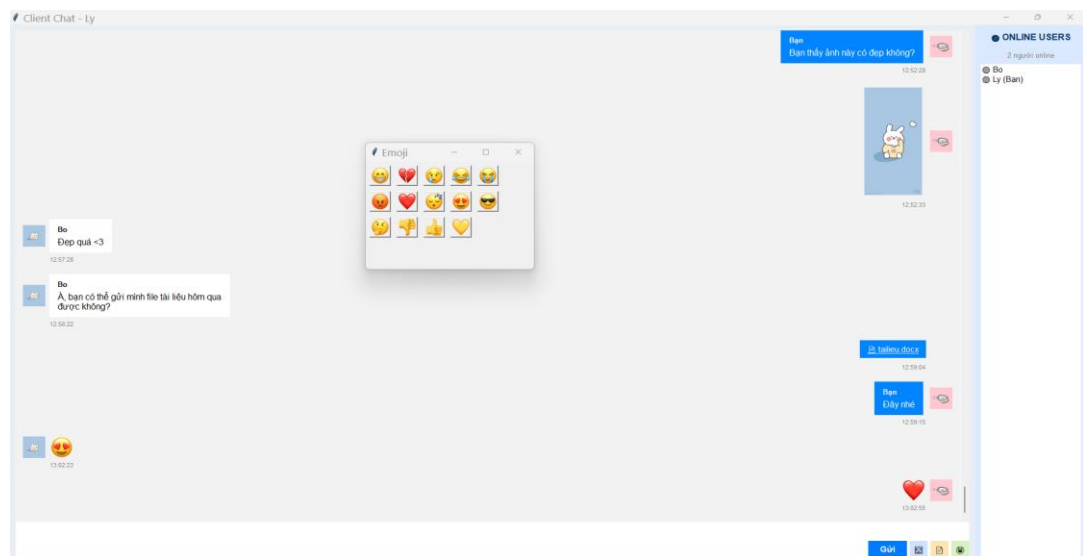


Hình 7.12: Kết quả tải file xuống

7.9 Kết quả gửi emoji

Hệ thống hỗ trợ gửi emoji từ thư mục emoji cục bộ. Sau khi gửi:

- Emoji hiển thị trực tiếp trên cửa sổ chat.
- Đồng bộ giữa các Client.
- Giao diện trực quan hơn.



Hình 7.13. Kết quả gửi emoji

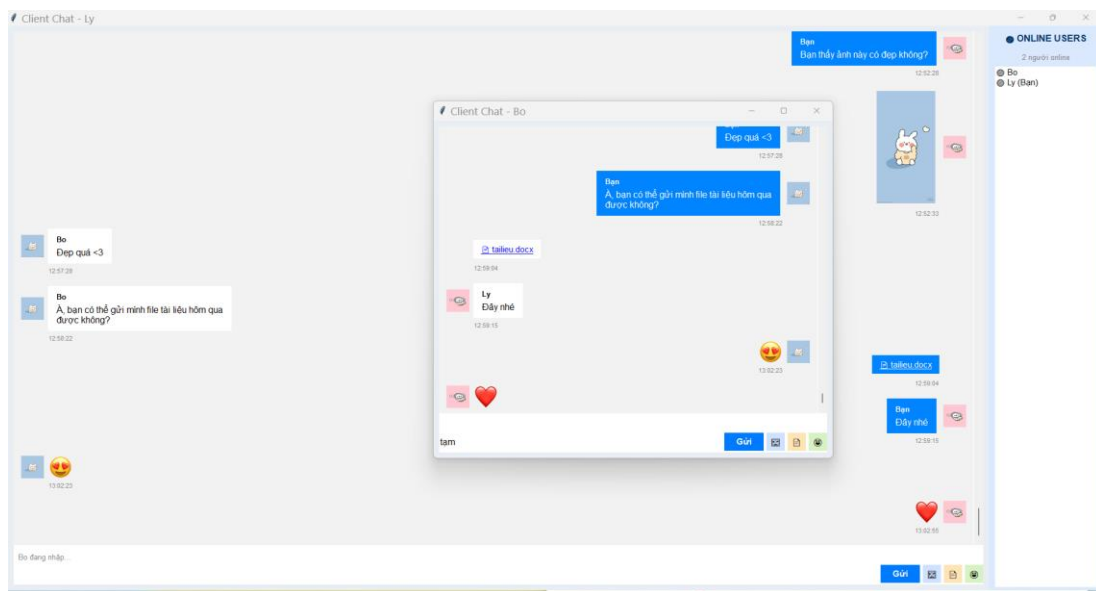
7.10 Kết quả hiển thị trạng thái đang nhập

Hệ thống hỗ trợ hiển thị trạng thái “đang nhập...” nhằm tăng tính tương tác giữa các người dùng. Khi người dùng bắt đầu nhập nội dung vào ô chat:

- Client gửi tín hiệu typing đến Server.
- Server chuyển tiếp đến các Client khác.
- Client nhận sẽ hiển thị trạng thái người gửi đang soạn tin.

Trạng thái này sẽ tự động ẩn sau một khoảng thời gian nếu không tiếp tục nhập nội dung. Chức năng này giúp:

- Tăng tính tương tác thời gian thực.
- Giúp người dùng biết đối phương đang trả lời.
- Giao diện giống các ứng dụng chat hiện đại.



Hình 7.14. Kết quả hiển thị trạng thái đang nhập

7.11 Kết quả chat riêng tư

Hệ thống hỗ trợ gửi tin nhắn riêng giữa hai người dùng thông qua cú pháp /private username message.

Tin nhắn chỉ hiển thị với người gửi và người nhận, không broadcast cho toàn bộ hệ thống.

Chức năng giúp trao đổi nội dung riêng tư trong cùng mạng LAN.

7.12 Đánh giá kết quả

Qua quá trình thử nghiệm cho thấy hệ thống hoạt động ổn định và đáp ứng được các chức năng chính:

- Kết nối nhiều Client đồng thời.
- Gửi nhận tin nhắn nhanh.
- Gửi file và ảnh thành công.
- Danh sách online cập nhật chính xác.
- Hoạt động tốt trong mạng LAN.
- Giao diện trực quan, dễ sử dụng.

Hệ thống đạt mục tiêu xây dựng ứng dụng chat cơ bản trong mạng LAN sử dụng giao thức TCP.

8. Ưu điểm

Hệ thống đạt được nhiều ưu điểm như:

- Giao diện trực quan, dễ sử dụng nhờ xây dựng bằng thư viện Tkinter.
- Hỗ trợ nhiều Client kết nối và chat cùng lúc thông qua xử lý đa luồng (Threading).
- Sử dụng giao thức TCP giúp dữ liệu truyền ổn định, chính xác và hạn chế mất gói tin.
- Có mã hóa dữ liệu bằng Fernet Encryption giúp tăng tính bảo mật khi truyền dữ liệu trong mạng LAN.
- Hỗ trợ tự động tìm Server bằng UDP Broadcast, giúp người dùng không cần nhập IP thủ công.
- Hỗ trợ gửi nhiều loại dữ liệu như:
 - Tin nhắn văn bản.
 - Hình ảnh.
 - File tài liệu.
 - Emoji.
- Có chức năng hiển thị trạng thái “đang nhập...” giúp tăng tính tương tác giữa người dùng.

- Hiện thị danh sách người dùng online theo thời gian thực và hỗ trợ avatar người dùng.
- Hỗ trợ hiện thị thời gian gửi tin nhắn và giao diện chat bubble hiện đại, dễ quan sát.
- Hệ thống hoạt động ổn định trong môi trường mạng LAN và dễ dàng nâng cấp thêm tính năng trong tương lai.

9. Nhược điểm

Bên cạnh các ưu điểm, hệ thống vẫn còn một số hạn chế như:

- Chưa sử dụng cơ sở dữ liệu nên thông tin người dùng và lịch sử chat chưa được lưu trữ lâu dài.
- Chưa hỗ trợ chức năng đăng nhập bằng tài khoản và mật khẩu nên tính bảo mật người dùng còn hạn chế.
- Ứng dụng hiện chỉ hoạt động trong mạng LAN, chưa hỗ trợ kết nối Internet public.
- Chưa tích hợp các chức năng nâng cao như voice call hoặc video call.
- Chưa áp dụng cơ chế mã hóa đầu cuối (End-to-End Encryption) nên dữ liệu vẫn phải giải mã tại Server trước khi gửi tiếp cho các Client khác.
- Chưa hỗ trợ tạo phòng chat hoặc chat nhóm riêng biệt giữa các người dùng.

10. Hướng phát triển

Trong tương lai, hệ thống có thể được nâng cấp và phát triển thêm nhiều chức năng nhằm tăng tính bảo mật, hiệu năng và trải nghiệm người dùng, bao gồm:

- Tích hợp cơ sở dữ liệu SQLite hoặc MySQL để lưu trữ tài khoản và lịch sử chat.
- Xây dựng chức năng đăng ký, đăng nhập và quản lý tài khoản người dùng.
- Áp dụng cơ chế mã hóa End-to-End để tăng tính bảo mật dữ liệu khi truyền tải.
- Hỗ trợ Voice Call và Video Call giữa các người dùng trong hệ thống.
- Phát triển Chat Room hoặc Group Chat để tạo nhiều phòng trò chuyện riêng biệt.
- Bổ sung chức năng thu hồi tin nhắn và hiện thị trạng thái “đã xem”.

- Thiết kế giao diện Dark Mode giúp tăng trải nghiệm sử dụng.
- Xây dựng Cloud Server để hỗ trợ kết nối qua Internet thay vì chỉ trong mạng LAN.
- Phát triển phiên bản Mobile App cho Android và iOS.
- Hỗ trợ gửi Sticker, GIF và nhiều định dạng media khác.
- Lưu lịch sử chat trực tiếp trên Server để đồng bộ dữ liệu giữa các thiết bị và người dùng.

11. Kết luận

Đề tài “Xây dựng ứng dụng chat trong mạng LAN sử dụng giao thức TCP” đã xây dựng thành công hệ thống chat hoạt động theo mô hình Client–Server bằng ngôn ngữ Python.

Hệ thống đáp ứng được các chức năng cơ bản của một ứng dụng chat hiện đại như:

- Chat thời gian thực.
- Gửi và nhận file.
- Gửi hình ảnh.
- Gửi emoji.
- Hiện thị trạng thái “đang nhập...”.
- Hiện thị người dùng online.
- Mã hóa dữ liệu bằng Fernet Encryption.
- Tự động tìm Server trong mạng LAN.

Thông qua quá trình thực hiện đề tài, giúp củng cố và nâng cao kiến thức về:

- Socket Programming.
- Giao thức TCP/IP và UDP.
- Xử lý đa luồng (Threading).
- Thiết kế giao diện GUI bằng Tkinter.
- Bảo mật dữ liệu trong ứng dụng mạng.

Mặc dù hệ thống vẫn còn một số hạn chế, đề tài đã tạo được nền tảng tốt để tiếp tục nghiên cứu và phát triển các ứng dụng chat và hệ thống mạng phức tạp hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Python Software Foundation, *Python Documentation*.
<https://docs.python.org/3/>
- [2] Python Software Foundation, *Socket Programming HOWTO*.
<https://docs.python.org/3/howto/sockets.html>
- [3] Python Software Foundation, *Tkinter Documentation*.
<https://docs.python.org/3/library/tkinter.html>
- [4] Pillow Team, *Pillow Documentation*.
<https://pillow.readthedocs.io/>
- [5] Cryptography Project, *Fernet Symmetric Encryption*.
<https://cryptography.io/en/latest/fernet/>
- [6] James F. Kurose, Keith W. Ross, *Computer Networking: A Top-Down Approach*.
Pearson.

PHỤ LỤC:

Phụ lục 1: Mã nguồn chương trình

GitHub Repository: <https://github.com/22T1020662/THUC-TAP-TOT-NGHIEP>